

Laboratorio di Algoritmi

Introduzione e Concetti Fondamentali

Martedì 3 Marzo 2026



Mappa del Corso: Informazioni Fondamentali



Docente: **Mariella Bonomo**

Valore: **6 CFU**



Martedì 14:30 – 17:30
(Lab. Biblioteca)

Giovedì 08:30 – 11:30
(Lab. Biblioteca)



Esami:
**Prova scritta +
progetto/prova orale**

Strumenti e Requisiti

Risorse Chiave



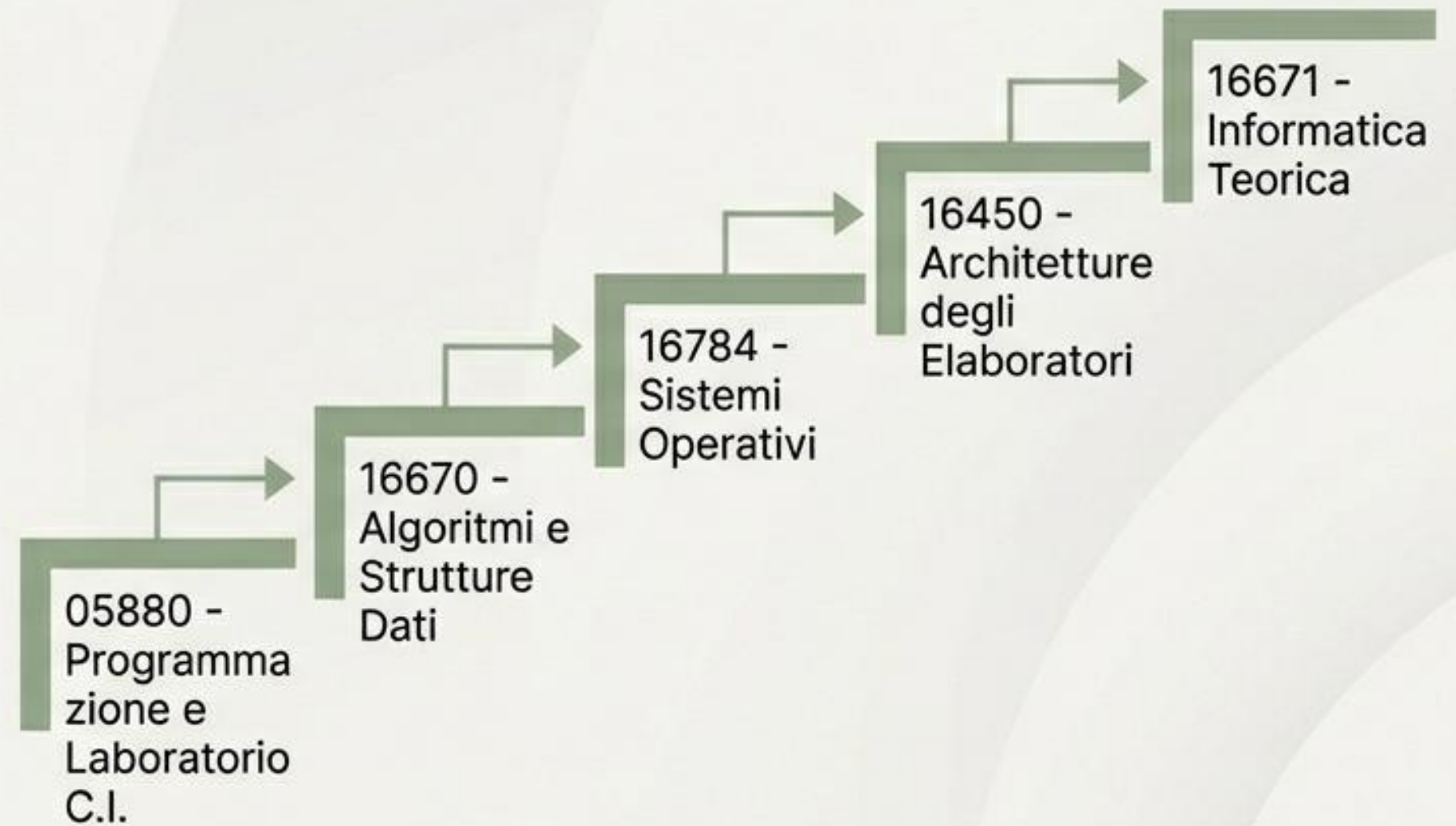
Testi di Riferimento

Cormen, Demetrescu,
Sedgewick, Ferragina



Risorsa Online Cruciale:
algs4.cs.princeton.edu

Propedeuticità



TESTI DI RIFERIMENTO E CONSULTAZIONE: VISIONARE LA SCHEDA DI TRASPARENZA

- The magic of Algorithms! Lectures on some algorithmic pearls. Paolo Ferragina, Università di Pisa. 2019.
- Introduzione agli algoritmi e strutture dati (3 ed). Cormen , Leiserson, Rivest, Stein. Mc Graw Hill, 2010 (Polinomi e FFT/Polynomials and the FFT)
- Algoritmi e Strutture Dati. Demetrescu, Finocchi, Italiano. Mc Graw Hill, 2008.
- Sedgewick & K. Wayne, Algorithms, 4th Edition, Addison-Wesley Professional.
- C. Demetrescu, U. Ferraro Petrillo, I. Finocchi, G. Italiano, Progetto di algoritmi e strutture dati in Java, Mc Graw Hill.

L'Obiettivo Centrale del Corso

ALGORITMI

Il “Come” risolvere un problema.

STRUTTURE DATI

Il “Come” memorizzare le informazioni.

AZIONE E IMPLEMENTAZIONE

Ripresa e approfondimento dei concetti teorici attraverso l'implementazione pratica.

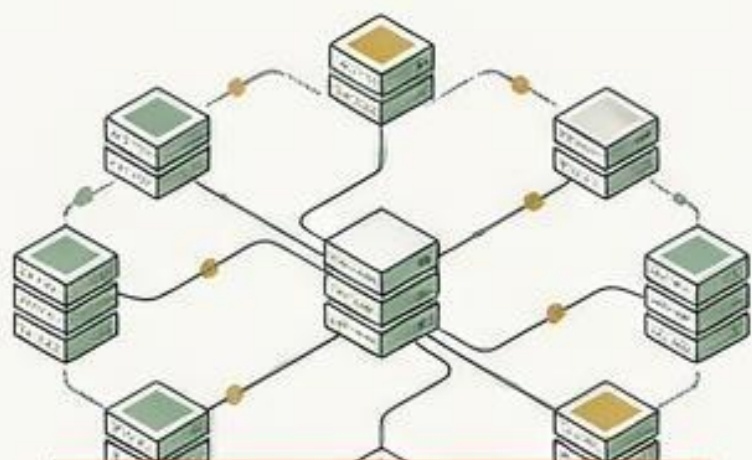


Algoritmo vs Programma

- Un **algoritmo** descrive (ad alto livello) una procedura di calcolo che, se seguita, consente di ottenere un certo risultato
- Un **programma** è l'implementazione di un algoritmo mediante un opportuno linguaggio di programmazione.
 - Un programma può essere direttamente eseguito da un calcolatore (processo in esecuzione); un algoritmo solitamente no.

Dove gli Algoritmi Fanno la Differenza

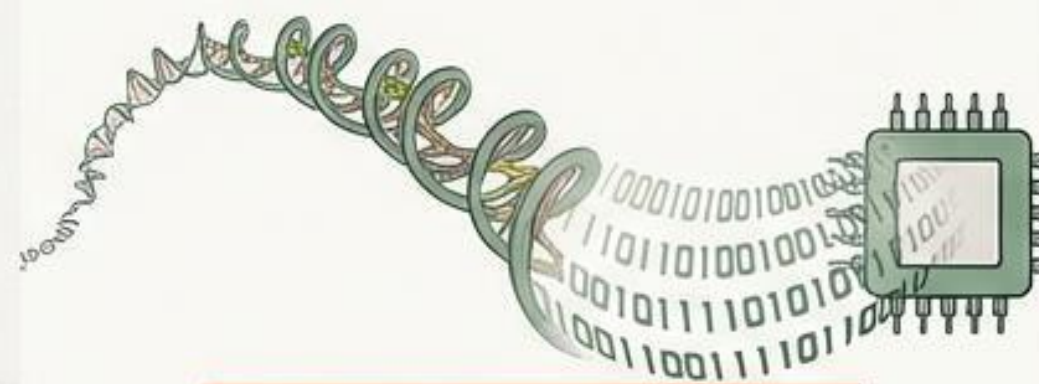
L'impatto di algoritmi efficienti è esteso e globale. Fare la differenza significa ottimizzare l'invisibile.



Internet



Social Network



Bioinformatica



Computer Graphics

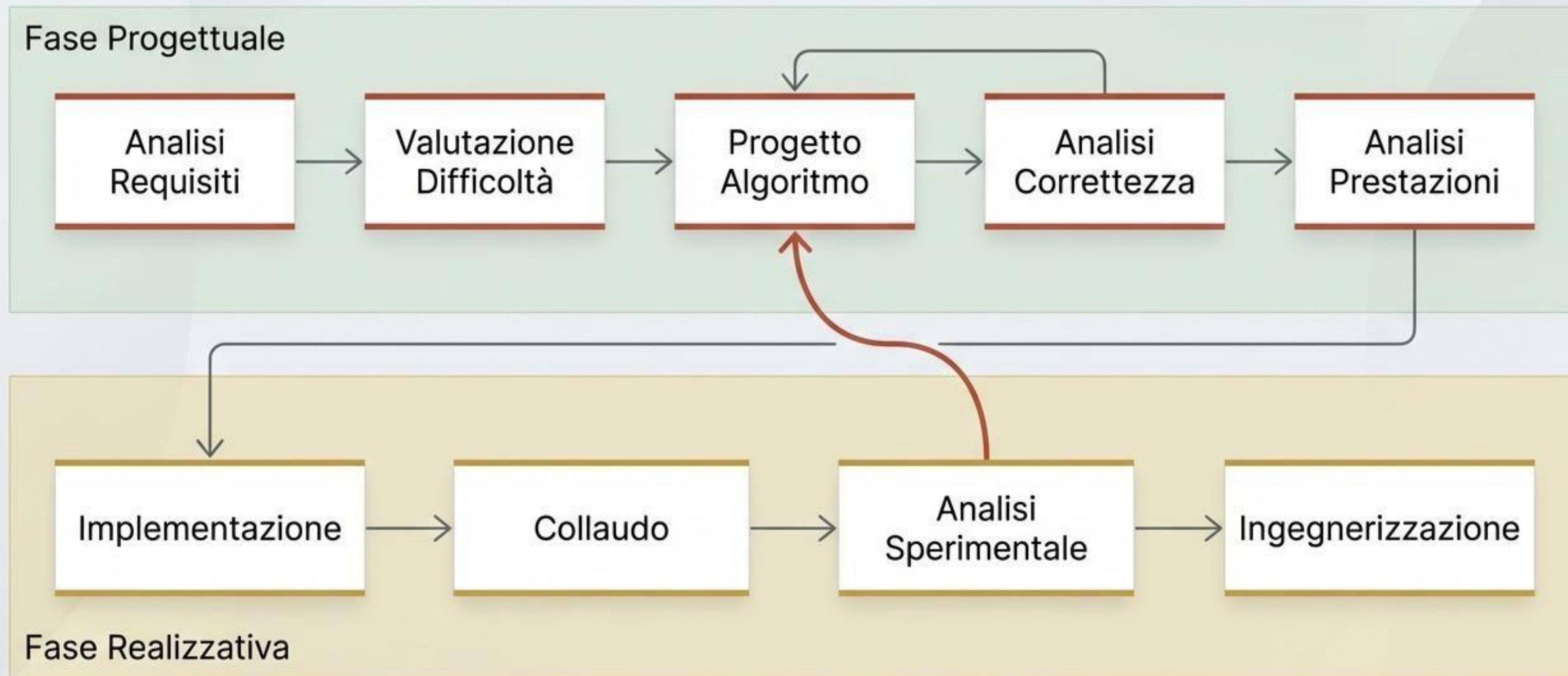


Sicurezza

“ I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.

– Linus Torvalds (Creatore di Linux)

Il Ciclo di Sviluppo di un Algoritmo



Deep Dive: La Fase Progettuale



1. Analisi & Difficoltà

Decomposizione in sottoproblemi. Definizione rigorosa di input e output. Studio dei limiti inferiori delle risorse.



2. Progetto & Correttezza

Ricerca della strategia con il minor uso di risorse. Dimostrazione formale: qualunque sia l'input, l'output è matematicamente corretto.



3. Prestazioni Teoriche

Modello di costo teorico e Notazione Asintotica. Analisi suddivisa per caso peggiore, caso migliore, caso medio, e analisi ammortizzata nel tempo.

Deep Dive: La Fase Realizzativa



1. Implementazione

La traduzione logica. La scelta del linguaggio di programmazione influenza drasticamente i tempi reali di esecuzione.



2. Collaudo Sperimentale

Identificazione e risoluzione di errori implementativi mediante l'esecuzione su dataset e scenari di test reali.



3. Ingegnerizzazione

Il passaggio finale. Trasformare un'idea ad alto livello in software robusto, modulare e riutilizzabile, sfruttando l'efficienza delle librerie di sistema.

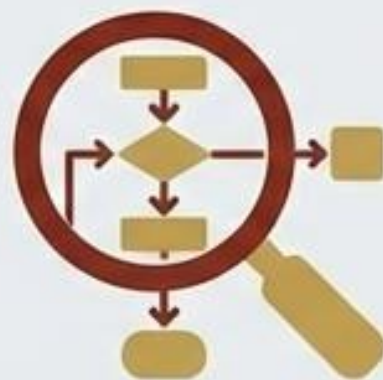
Misurare le Risorse: Il Tempo

L'Obiettivo Primario: Minimizzare le risorse (Tempo e Spazio di Memoria).



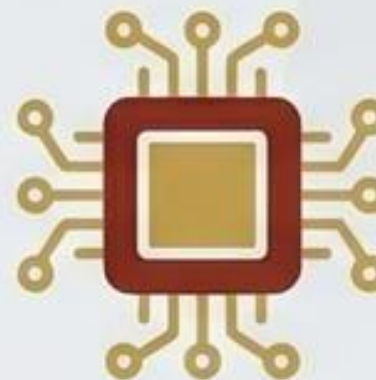
Wall-clock time

Il tempo fisico effettivo trascorso sull'orologio.



Operazioni rilevanti

Conteggio dei passaggi logici chiave e determinanti dell'algoritmo.



Operazioni elementari

Il costo computazionale misurato al livello più basso della macchina.

Cosa imparerete in questo corso?

- Quali sono gli algoritmi “classici” per risolvere problemi ricorrenti
 - Ordinamento, ricerca, problemi su grafi...
- Come valutare l'efficienza di un algoritmo in termini di prestazioni ed utilizzo di risorse
- Come sviluppare nuovi algoritmi
- Limiti

Valutazione degli algoritmi: altre proprietà

- Correttezza, Semplicità, Modularità, Manutenibilità,
- Espandibilità, Sicurezza, Robustezza, User- Friendly..
- Trade-off tra prestazioni e modularità
- Trade-off tra prestazioni e

Problema computazionale: esempi

♦Minimo

♦Il minimo di un insieme A è l'elemento di A che è minore o uguale ad ogni elemento di A

$$\min(A) = a \Leftrightarrow \forall b \in A : a \leq b$$

♦Ricerca

♦Sia $A=a_1, \dots, a_n$ una sequenza di dati ordinati e distinti, $a_1 < a_2 < \dots < a_n$. Eseguire una ricerca della posizione di un dato v in A consiste nel restituire l'indice corrispondente, se v è presente, oppure 0, se v non è presente

$$ricerca(A, v) = \begin{cases} i & \exists i \in \{1, \dots, n\} : a_i = v \\ 0 & \text{altrimenti} \end{cases}$$

COMINCIAMO CON UN ESEMPIO SEMPLICE

- Problema del minimo in un vettore: misuriamo il numero di confronti
- Algoritmo ingenuo

```
min( $S, n$ )  
for  $i = 1$  to  $n$  do  
     $isMin = \text{true}$   
    for  $j = 1$  to  $n$  do  
        if  $S[j] < S[i]$  then  
             $isMin = \text{false}$   
    if  $isMin$  then  
        return  $S[i]$ 
```

- Complessità di tempo: n^2 confronti

MIGLIORIAMO LA SOLUZIONE PER TROVARE IL MINIMO

```
min( $S, n$ )  
 $min = S[1]$   
for  $i = 2$  to  $n$  do  
    if  $S[i] < min$  then  
         $min = S[i]$   
return  $min$ 
```

Minimo parziale

Nuovo minimo
parziale

- Complessità di tempo: $n-1$ confronti
- E' un algoritmo ottimale

```
lookup( $S, n, v$ )
```

```
for  $i = 1$  to  $n$  do  
    if  $S[i] = v$  then  
        return  $i$ 
```

```
return 0
```

-
- Algoritmo ingenuo: n confronti

UNA SEQUENZA ORDINATA

ALTRO PROBLEMA SEMPLICE: RICERCA IN

RICERCA BINARIA

- E' una soluzione più efficiente
- Richiede al più $\lfloor \log n \rfloor + 1$ confronti
- Perché?

Ricerca in un array ordinato

ITEM **binarySearch**(ITEM[] *A*, ITEM *v*, **integer** *i*, **integer** *j*)

if $i > j$ **then**

 | **return** 0

else

 | **integer** $m \leftarrow \lfloor (i + j) / 2 \rfloor$

 | **if** $A[m] = v$ **then**

 | **return** m

 | **else if** $A[m] < v$ **then**

 | **return** **binarySearch**($A, v, m + 1, j$)

 | **else**

 | **return** **binarySearch**($A, v, i, m - 1$)

Ricerca in un array ordinato

✦ Problema

- ✦ Dato un vettore A contenente n elementi, verificare se un certo elemento v è presente
- ✦ Esempi: elenco del telefono, dizionario

✦ Una soluzione “banale”

- ✦ Scorro gli elementi in ordine, finché non trovo un oggetto “maggiore o uguale” a v

1	5	12	15	20	23	32
---	---	----	----	----	----	----

~~21~~

Valutazione degli algoritmi

Ricerca del minimo in un vettore;

Ricerca all'interno di un vettore di un valore v ;

Quanti confronti?

$(n-1)$

(n)

Perché contare i confronti?

Valutazione degli algoritmi: array ordinato

Contiamo il numero di confronti:

Ad ogni confronto dimezziamo lo spazio di ricerca:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{2^2} \rightarrow \frac{n}{2^3} \dots \dots \rightarrow \dots \dots \frac{n}{2^k}$$

Complessità in tempo

Complessità in tempo: cosa serve?

Per stimare il tempo impiegato da un programma

Per stimare il più grande input gestibile in tempi ragionevoli

Per confrontare l'efficienza di algoritmi diversi

Per ottimizzare le parti più importanti

“Complessità”: “Dimensione” $\rightarrow$ “Tempo”

Dobbiamo definire “dimensione” e “tempo”!

Complessità in tempo

- Tempo di calcolo di `min()`

integer `min`(ITEM[] *A*, **integer** *n*)

ITEM *min* $\leftarrow A[1]$

for integer *i* $\leftarrow 2$ **to** *n* **do**

if $A[i] < min$ **then**

$min \leftarrow A[i]$

return *min*

Costo # Volte

c_1 1

c_2 n

c_3 $n - 1$

c_4 $n - 1$

c_5 1

Complessità in tempo

- Tempo di calcolo di `min()`

$$\begin{aligned}T(n) &= c_1 + c_2n + c_3(n-1) + c_4(n-1) + c_5 = \\&= (c_2 + c_3 + c_4)n + (c_1 + c_5 - c_3 - c_4) \\&= an + b\end{aligned}$$

CONSIDERIAMO UN ALTRO PROBLEMA...

- Nell'ambito dell'analisi delle sequenze genomiche, un problema di ricerca consiste nel localizzare le regioni biologicamente significative, come le regioni GC-rich. Un approccio comune consiste nell'assegnare uno score ad ogni nucleotide, per poi cercare il segmento di somma massima.
- Problema: cercare in un array la sottosequenza di somma massimale
 - Input: una sequenza di interi $A[1...n]$
 - Output: la sottosequenza $A[i \dots j]$ di somma massimale, ovvero la sottosequenza la cui somma degli elementi dalla posizione i alla j è maggiore o uguale alla somma degli elementi di una qualsiasi altra sottosequenza

1	3	4	-9	2	3	-1	3	4	-3	10	-3	2
---	---	---	----	---	---	----	---	---	----	----	----	---

Caso di Studio: La Sottosequenza Massimale



Il Problema: Dato un array di N numeri reali, trovare la somma massima in una sottosequenza contigua.

I Casi Limite

- **E se sono tutti positivi?** La soluzione è l'array intero.
- **E se sono tutti negativi?** La soluzione è l'array vuoto (somma = 0).

LA SOLUZIONE PIÙ SEMPLICE

```
MaxSoFar := 0.0
for L := 1 to N do
  for U := L to N do
    Sum := 0.0
    for I := L to U do
      Sum := Sum + X[I]
    /* Sum now contains the
       sum of X[L..U] */
    MaxSoFar := max(MaxSoFar, Sum)
```

- E' un tempo di $O(n^3)$
- ESERCIZIO: Implementare in C la funzione che preso in input un vettore di N numeri reali generati casualmente calcolo la somma della sottosequenza massimale. Misurare il tempo di calcolo in secondi per valori di N pari a 100, 1000, 2000, 5000, 10000.

CERCHIAMO SOLUZIONI PIÙ EFFICIENTI

- Si nota che la somma $X[L..U]$ si ottiene da $X[L..U-1] + X[U]$

```
MaxSoFar := 0.0
for L := 1 to N do
  Sum := 0.0
  for U := L to N do
    Sum := Sum + X[U]
    /* Sum now contains the
       sum of X[L..U] */
    MaxSoFar := max(MaxSoFar, Sum)
```

Complessità di
tempo quadratica

1	3	4	-9	2	3	-1	3	4	-3	10	-3	2
1	4	8	-1	1	4	3	6	10	7	17	14	16
	3	7	-2	0	3	2	5	9	6	16	13	15
		4	-5	-3	0	-1	2	6	3	13	10	12

- Implementare la soluzione e confrontare i tempi con la soluzione precedente

USO DI ARRAY CUMULATIVI

- Un'altra soluzione computa la somma nel ciclo interno accedendo ad una struttura dati (chiamata CumArray) costruita prima che il ciclo esterno venga eseguito. L'elemento h-esimo di CumArray contiene la somma dei valori di $X[1..h]$, cosicchè la somma dei valori in $X[L..U]$ si trova computando $\text{CumArray}[U] - \text{CumArray}[L-1]$

```
CumArray[0] := 0.0
for I := 1 to N do
    CumArray[I] := CumArray[I - 1] + X[I]
MaxSoFar := 0.0
for L := 1 to N do
    for U := L to N do
        Sum := CumArray[U] - CumArray[L - 1]
        /* Sum now contains the
           sum of X[L..U] */
        MaxSoFar := max(MaxSoFar, Sum)
```

- Qual è la complessità?
 - Quadratica!

Il Potere dell'Ottimizzazione



39 GIORNI

Algoritmo Ingenuo



0.3 SECONDI

Algoritmo Ottimizzato

Contesto: Jon Bentley (1984). Risoluzione dello stesso identico problema (Sottosequenza Massimale) con dimensione $N = 10.000$. Questo è il motivo per cui studiamo gli algoritmi.

L'Evoluzione della Soluzione



1. Algoritmo Ingenuo

Complessità temporale n^2 . Doppio ciclo annidato. Costo computazionale enorme per via dei troppi confronti.



2. Dividi et Impera

Scomposizione del vettore. Calcolo di Max_A (sinistra) e Max_B (destra), richiedendo però attenzione a Max_C (la sottosequenza che attraversa il taglio centrale).

3. L'Approccio Lineare

Estensione iterativa ed elegante. Calcola la somma massima totale finora (MaxSoFar) e la somma massima che termina nella posizione corrente (MaxEndingHere).

La Sfida di Laboratorio



Esercizio Pratico: Implementare in C o Java le funzioni per la ricerca della Sottosequenza Massimale.

Input: Vettori di N numeri reali generati casualmente.

Wall-clock time
(secondi)

N = 100 1000 2000 5000 10.000

Obiettivo Finale: Misurare ed eseguire il benchmark per confrontare i tempi reali con la complessità teorica studiata. L'inizio pratico del corso.